```
"""
Automated EDA Report
  1. Statistical summaries and visualizations
  2. Correlations and key influencing factors (drivers of a target column)
  3. Structured report (self-contained HTML)

Usage:
    python eda_report.py                          # demo data
    python eda_report.py data.csv                 # your CSV (target auto-picked)
    python eda_report.py data.csv target_column   # your CSV + chosen target

Needs: pandas, numpy, matplotlib, seaborn
"""

import io
import sys
import base64
import datetime as dt

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid", palette="deep")


# ---------------------------------------------------------------------
# DATA
# ---------------------------------------------------------------------
def make_demo_data(n=600, seed=7):
    """Messy demo data where spend_score truly depends on income, age, segment."""
    rng = np.random.default_rng(seed)
    age = rng.normal(38, 11, n).clip(18, 75).round()
    income = rng.normal(55000, 14000, n).clip(15000, None).round(2)
    segment = rng.choice(["Basic", "Plus", "Premium"], n, p=[.5, .3, .2])
    region = rng.choice(["North", "South", "East", "West"], n)
    bonus = pd.Series(segment).map({"Basic": 0, "Plus": 8, "Premium": 18}).to_numpy()
    spend = (20 + income / 2500 - 0.2 * (age - 38) + bonus + rng.normal(0, 6, n)).clip(1,
100).round(1)

    df = pd.DataFrame({
        "customer_id": np.arange(1, n + 1), "age": age, "income": income,
        "segment": segment, "region": region, "spend_score": spend,
    })
    for col, frac in [("age", .07), ("income", .09), ("region", .05)]:
        df.loc[rng.choice(n, int(n * frac), replace=False), col] = np.nan
    df.loc[rng.choice(n, 5, replace=False), "income"] = [400000, 650000, 380000, -20000, 500000]
    df.loc[rng.choice(n, 3, replace=False), "age"] = [150, 140, -5]
    return pd.concat([df, df.sample(20, random_state=1)], ignore_index=True)
```

```
# ---------------------------------------------------------------------
# CLEANING (duplicates -> missing -> outliers)
# ---------------------------------------------------------------------
def clean(df):
    log = {"rows_raw": len(df), "cols": df.shape[1], "missing_raw": int(df.isna().sum().sum()),
           "missing_by_col": df.isna().sum()[df.isna().sum() > 0].to_dict()}

    log["duplicates"] = int(df.duplicated().sum())
    df = df.drop_duplicates().reset_index(drop=True)

    id_cols = [c for c in df.select_dtypes("number").columns if df[c].is_unique]
    num = [c for c in df.select_dtypes("number").columns if c not in id_cols]
    cat = [c for c in df.select_dtypes(exclude="number").columns
           if not pd.api.types.is_datetime64_any_dtype(df[c])]

    for c in num:
        if df[c].isna().any():
            df[c] = df[c].fillna(df[c].median() if abs(df[c].skew()) > 1 else df[c].mean())
    for c in cat:
        if df[c].isna().any():
            df[c] = df[c].fillna(df[c].mode().iloc[0])

    log["outliers"] = {}
    for c in num:
        q1, q3 = df[c].quantile([.25, .75])
        lo, hi = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
        n_out = int(((df[c] < lo) | (df[c] > hi)).sum())
        if n_out:
            log["outliers"][c] = n_out
        df[c] = df[c].clip(lo, hi)

    log["rows_clean"] = len(df)
    return df, id_cols, num, cat, log


# ---------------------------------------------------------------------
# 1. STATISTICAL SUMMARIES
# ---------------------------------------------------------------------
def numeric_summary(df, num):
    s = df[num].describe().T[["count", "mean", "std", "min", "25%", "50%", "75%", "max"]]
    s["skew"] = df[num].skew()
    s["kurtosis"] = df[num].kurt()
    return s.round(2)


def categorical_summary(df, cat):
    rows = []
    for c in cat:
        vc = df[c].value_counts()
        rows.append({"column": c, "unique": df[c].nunique(), "top": vc.index[0],
                     "top share": f"{vc.iloc[0] / len(df):.0%}"})
    return pd.DataFrame(rows).set_index("column") if rows else pd.DataFrame()


# ---------------------------------------------------------------------
# 2. CORRELATIONS AND KEY DRIVERS
# ---------------------------------------------------------------------
def strength(v):
```

```
    v = abs(v)
    return "strong" if v >= .6 else "moderate" if v >= .3 else "weak" if v >= .1 else "negligible"


def key_drivers(df, target, id_cols):
    """Numeric features: Pearson r.  Categorical features: correlation ratio (eta)."""
    rows = []
    y = df[target]
    ss_total = ((y - y.mean()) ** 2).sum()
    for c in df.columns:
        if c == target or c in id_cols or pd.api.types.is_datetime64_any_dtype(df[c]):
            continue
        if pd.api.types.is_numeric_dtype(df[c]):
            r = df[c].corr(y)
            rows.append((c, "numeric", "Pearson r", r, abs(r)))
        else:
            ss_between = sum(len(g) * (g.mean() - y.mean()) ** 2 for _, g in df.groupby(c)[target])
            eta = float(np.sqrt(ss_between / ss_total)) if ss_total else 0.0
            rows.append((c, "categorical", "Correlation ratio", eta, eta))
    out = pd.DataFrame(rows, columns=["feature", "type", "measure", "value", "strength"])
    out["label"] = out["strength"].map(strength)
    return out.sort_values("strength", ascending=False).reset_index(drop=True)


# ---------------------------------------------------------------------
# VISUALIZATIONS
# ---------------------------------------------------------------------
def to_img(fig, alt):
    buf = io.BytesIO()
    fig.savefig(buf, format="png", dpi=110, bbox_inches="tight")
    plt.close(fig)
    return f'<img alt="{alt}" src="
{base64.b64encode(buf.getvalue()).decode()}">'


def grid(n, ncols=3, w=5, h=3.6):
    nrows = int(np.ceil(n / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(w * ncols, h * nrows))
    axes = np.atleast_1d(axes).ravel()
    for ax in axes[n:]:
        ax.axis("off")
    return fig, axes


def charts(df, num, cat, target, drivers):
    out = {}

    fig, axes = grid(len(num))
    for ax, c in zip(axes, num):
        sns.histplot(df[c], kde=True, ax=ax)
        ax.set_title(c)
    fig.tight_layout()
    out["dist"] = to_img(fig, "Distributions")

    fig, axes = grid(len(num))
    for ax, c in zip(axes, num):
        sns.boxplot(y=df[c], ax=ax, color="#8fb8de")
        ax.set_title(c)
```

```
    fig.tight_layout()
    out["box"] = to_img(fig, "Boxplots")

    if cat:
        fig, axes = grid(len(cat))
        for ax, c in zip(axes, cat):
            sns.countplot(data=df, x=c, order=df[c].value_counts().index, ax=ax)
            ax.set_title(c)
            ax.tick_params(axis="x", rotation=25)
        fig.tight_layout()
        out["cat"] = to_img(fig, "Category counts")

    if len(num) > 1:
        fig, ax = plt.subplots(figsize=(7, 5.5))
        sns.heatmap(df[num].corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0, ax=ax)
        out["heat"] = to_img(fig, "Correlation heatmap")

    fig, ax = plt.subplots(figsize=(8, max(2.5, 0.55 * len(drivers))))
    colors = ["#29a36a" if (r.type == "numeric" and r.value > 0) else
              "#e4572e" if r.type == "numeric" else "#2a6fdb" for r in drivers.itertuples()]
    ax.barh(drivers["feature"][::-1], drivers["strength"][::-1], color=colors[::-1])
    ax.set_xlabel(f"Strength of relationship with {target} (0 to 1)")
    ax.set_title("Key influencing factors")
    out["drivers"] = to_img(fig, "Key drivers")

    top = drivers.iloc[0]
    fig, ax = plt.subplots(figsize=(7, 4.5))
    if top.type == "numeric":
        sns.regplot(data=df, x=top.feature, y=target, ax=ax, scatter_kws={"alpha": .4, "s": 18})
    else:
        sns.boxplot(data=df, x=top.feature, y=target, ax=ax)
    ax.set_title(f"{target} vs {top.feature} (top driver)")
    out["top"] = to_img(fig, "Top driver")
    return out


# ---------------------------------------------------------------------
# 3. STRUCTURED REPORT
# ---------------------------------------------------------------------
def insights(drivers, target, df):
    L = []
    top = drivers.head(3)
    L.append("Top factors for <b>{}</b>: ".format(target) + ", ".join(
        f"<b>{r.feature}</b> ({r.label}, {r.value:+.2f})" if r.type == "numeric"
        else f"<b>{r.feature}</b> ({r.label}, {r.value:.2f})" for r in top.itertuples()) + ".")
    for r in top.itertuples():
        if r.type == "numeric" and r.strength >= .1:
            d = "higher" if r.value > 0 else "lower"
            L.append(f"Higher <b>{r.feature}</b> goes with {d} {target}.")
        elif r.type == "categorical" and r.strength >= .1:
            g = df.groupby(r.feature)[target].mean().sort_values()
            L.append(f"By <b>{r.feature}</b>, average {target} is highest for <b>{g.index[-1]}</b> "
                     f"({g.iloc[-1]:.1f}) and lowest for <b>{g.index[0]}</b> ({g.iloc[0]:.1f}).")
    weak = drivers[drivers.strength < .1].feature.tolist()
    if weak:
        L.append("Little to no relationship: " + ", ".join(f"<b>{w}</b>" for w in weak) + ".")
    L.append("Correlation shows association, not cause. Confirm with domain knowledge or an
experiment.")
```

```
    return L


CSS = """
body{font-family:-apple-system,Segoe UI,Roboto,sans-
serif;margin:0;background:#f5f7fb;color:#1c2430;line-height:1.55}
main{max-width:980px;margin:auto;padding:20px}
h1{margin:.2em 0}h2{border-bottom:2px solid #2a6fdb;padding-bottom:4px;margin-top:2em;color:#2a6fdb}
.card{background:#fff;border-radius:10px;padding:14px 18px;margin:12px 0;box-shadow:0 1px 4px #0001}
.kpis{display:flex;flex-wrap:wrap;gap:12px}.kpi{flex:1 1 140px;background:#fff;border-
radius:10px;padding:12px;text-align:center;box-shadow:0 1px 4px #0001}
.kpi b{display:block;font-size:1.6em;color:#2a6fdb}
img{max-width:100%;height:auto}.scroll{overflow-x:auto}
table{border-collapse:collapse;font-size:.9em;width:100%}th,td{padding:6px 10px;border-bottom:1px
solid #e3e8f0;text-align:right}
th:first-child,td:first-child{text-align:left}th{background:#eef3fb}
"""


def build_report(df, num, cat, target, log, nsum, csum, drivers, imgs, path="eda_report.html"):
    kp = [("Rows (clean)", f"{log['rows_clean']:,}"), ("Duplicates removed", log["duplicates"]),
          ("Missing cells fixed", log["missing_raw"]), ("Outliers capped",
sum(log["outliers"].values()))]
    kpis = "".join(f"<div class='kpi'><b>{v}</b>{k}</div>" for k, v in kp)
    ins = "".join(f"<li>{t}</li>" for t in insights(drivers, target, df))
    miss = ", ".join(f"{k} ({v})" for k, v in log["missing_by_col"].items()) or "none"
    outl = ", ".join(f"{k} ({v})" for k, v in log["outliers"].items()) or "none"
    dtbl =
drivers.assign(value=drivers["value"].round(3)).drop(columns="strength").to_html(index=False,
border=0)

    sec = lambda t, body: f"<h2>{t}</h2>{body}"
    figs = lambda *k: "".join(f"<div class='card'>{imgs[x]}</div>" for x in k if x in imgs)

    page = f"""<!doctype html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1"><title>EDA Report</title>
<style>{CSS}</style></head><body><main>
<h1>Exploratory Data Analysis Report</h1>
<p>Target of interest: <b>{target}</b> · Generated {dt.date.today():%d %b %Y}</p>
<div class="kpis">{kpis}</div>

{sec("1. Executive summary", f"<div class='card'><ul>{ins}</ul></div>")}

{sec("2. Data quality", f"<div class='card'><ul><li>Missing values filled: {miss}</li>"
     f"<li>Outliers capped (IQR rule): {outl}</li><li>Duplicate rows removed: {log['duplicates']}
</li></ul></div>")}

{sec("3. Statistical summary", f"<div class='card scroll'><h3>Numeric</h3>{nsum.to_html(border=0)}"
     + (f"<h3>Categorical</h3>{csum.to_html(border=0)}" if len(csum) else "") + "</div>")}

{sec("4. Visualizations", figs("dist", "box", "cat"))}

{sec("5. Correlations", figs("heat"))}

{sec("6. Key influencing factors", figs("drivers", "top") +
     f"<div class='card scroll'>{dtbl}</div><p><small>Numeric features use Pearson r (-1 to 1). "
     "Categorical features use the correlation ratio (0 to 1).</small></p>")}
```

```
{sec("7. Recommendations", "<div class='card'><ul><li>Focus on the top-ranked factors when planning
actions.</li>"
     "<li>Investigate negligible factors before spending effort on them.</li>"
     "<li>Re-run this report when new data arrives to track changes.</li></ul></div>")}
</main></body></html>"""
    with open(path, "w", encoding="utf-8") as f:
        f.write(page)
    print(f"[Report] saved to {path}")


# ---------------------------------------------------------------------
# RUN
# ---------------------------------------------------------------------
def main():
    raw = pd.read_csv(sys.argv[1]) if len(sys.argv) > 1 else make_demo_data()
    df, id_cols, num, cat, log = clean(raw)

    target = sys.argv[2] if len(sys.argv) > 2 else (
        "spend_score" if "spend_score" in num else num[-1])
    if target not in df.columns:
        sys.exit(f"Target '{target}' not found. Columns: {list(df.columns)}")

    nsum, csum = numeric_summary(df, num), categorical_summary(df, cat)
    drivers = key_drivers(df, target, id_cols)
    imgs = charts(df, num, cat, target, drivers)

    print(drivers.round(3).to_string(index=False))
    df.to_csv("clean_data.csv", index=False)
    build_report(df, num, cat, target, log, nsum, csum, drivers, imgs)


if __name__ == "__main__":
    main()
```